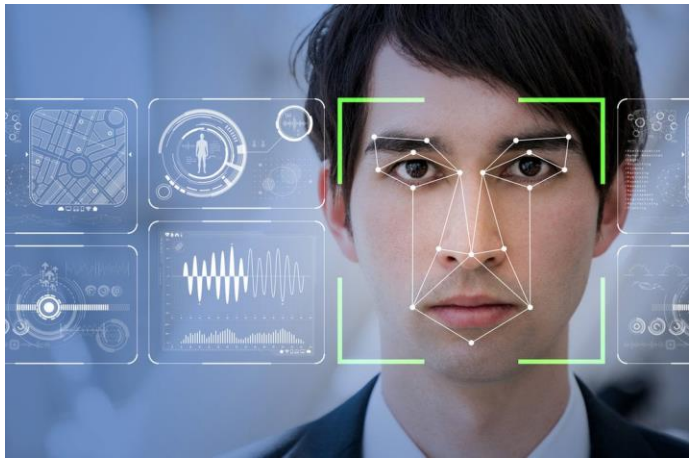
Statistical Machine Learning

#2 – Artificial Neural Network

Artificial Neural Network

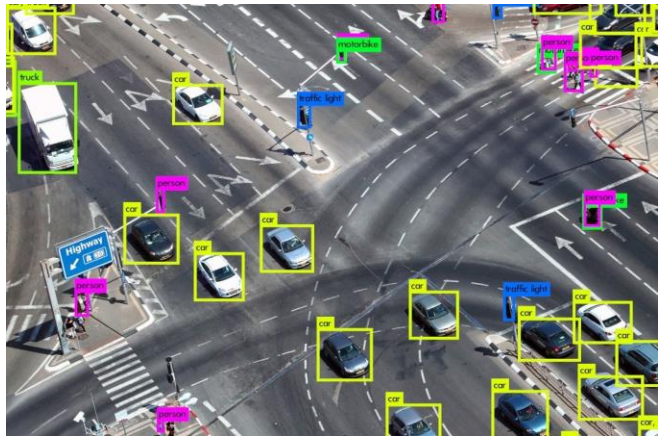
- Definisi
- Layer
- Fitting
- Examples

Beberapa Kegunaan



Face Recognition,
Pattern Recognition

Signal Processing,
Natural Language
Processing



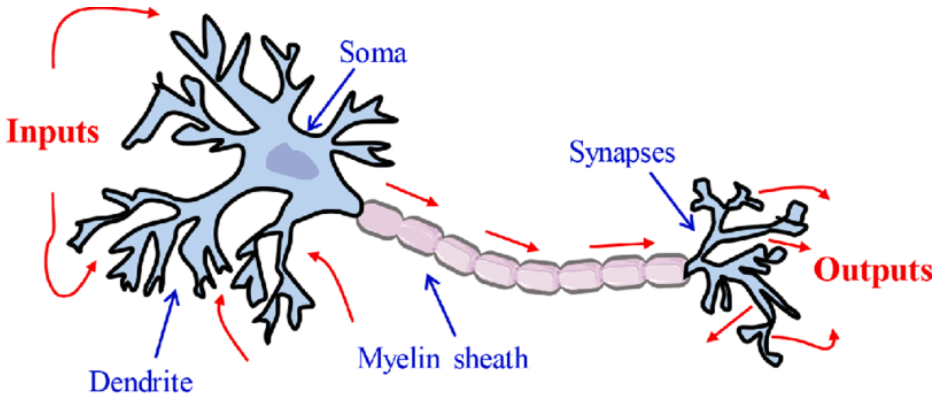
Kontrol dan Pengawasan

Klasifikasi,
Prediksi

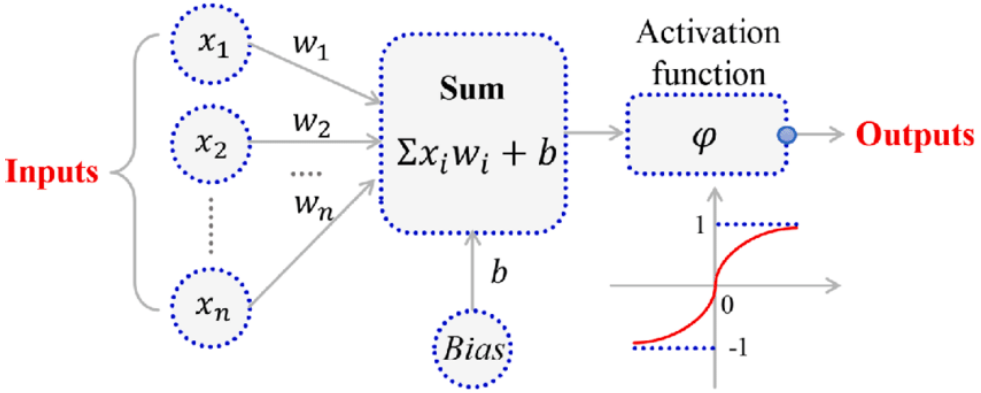
Pendahuluan

- Artificial Neural Network (ANN), atau Jaringan Saraf Tiruan, adalah sebuah model komputasi yang terinspirasi dari cara kerja otak manusia dalam memproses informasi.
- ANN terdiri dari banyak unit pemrosesan (neuron) yang saling terhubung dan mampu belajar dari data masukan yang diberikan.
- Setiap unit pemrosesan atau neuron dalam ANN memiliki fungsi matematika yang digunakan untuk menghitung keluaran berdasarkan masukan yang diterimanya.
- Koneksi antar-neuron dalam ANN memiliki bobot atau nilai yang mempengaruhi kontribusi setiap neuron pada proses pemrosesan data.

Ide Dasar



(a) Biological neuron



(b) Artificial neuron

<u>Inspirasi dari Otak</u>

Otak manusia memiliki **neuron** yang saling terhubung. Setiap neuron menerima **input**, memprosesnya, dan meneruskan hasilnya ke neuron lain.

<u>ANN Meniru Ini</u>

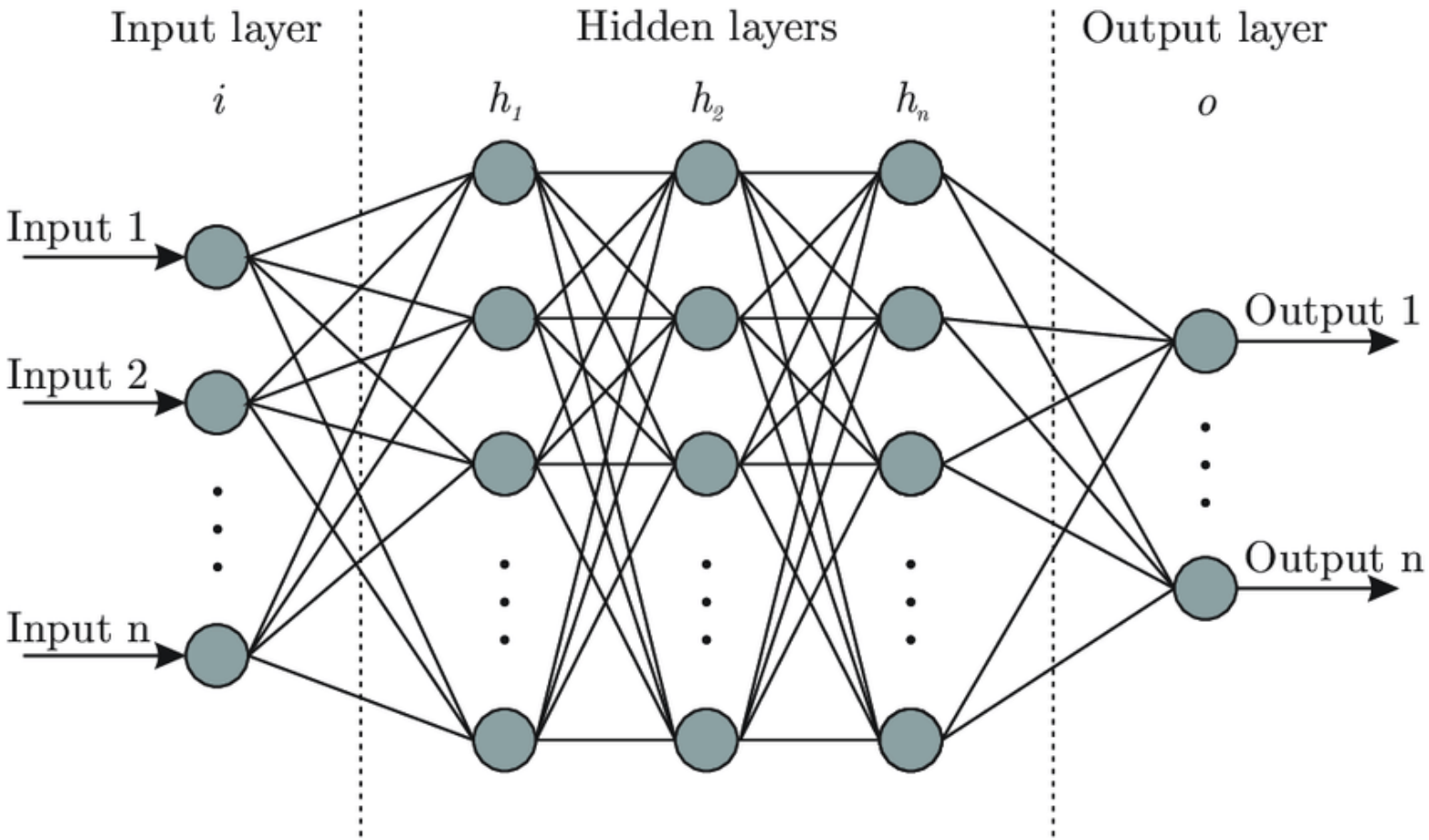
Neuron dalam ANN menerima **bobot (weights)** yang menentukan seberapa kuat pengaruh sebuah fitur. Bobot ini **dioptimalkan** selama training agar model belajar **pola terbaik** dalam data.

Artificial Neural Network

"A neural network is a two-stage regression or classification model, typically represented by a network diagram. This network applies both to regression or classification. For regression, typically K = 1 and there is only one output unit Y1 at the top. However, these networks can handle multiple quantitative responses in a seamless fashion, so we will deal with the general case"

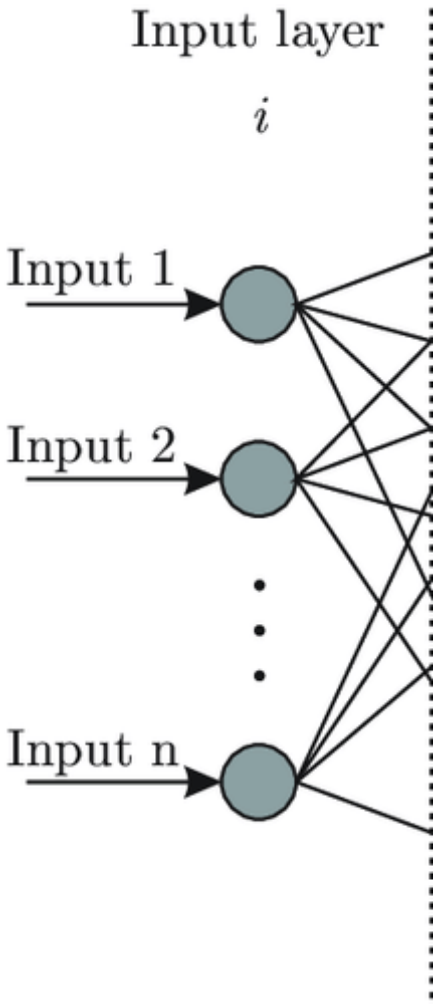
- Struktur ANN
  - Input Layer → Menerima fitur (variabel independen) dari data.
  - Hidden Layers → Mentransformasi fitur untuk menemukan pola.
  - Output Layer → Menghasilkan prediksi (bisa regresi atau klasifikasi).

Layer

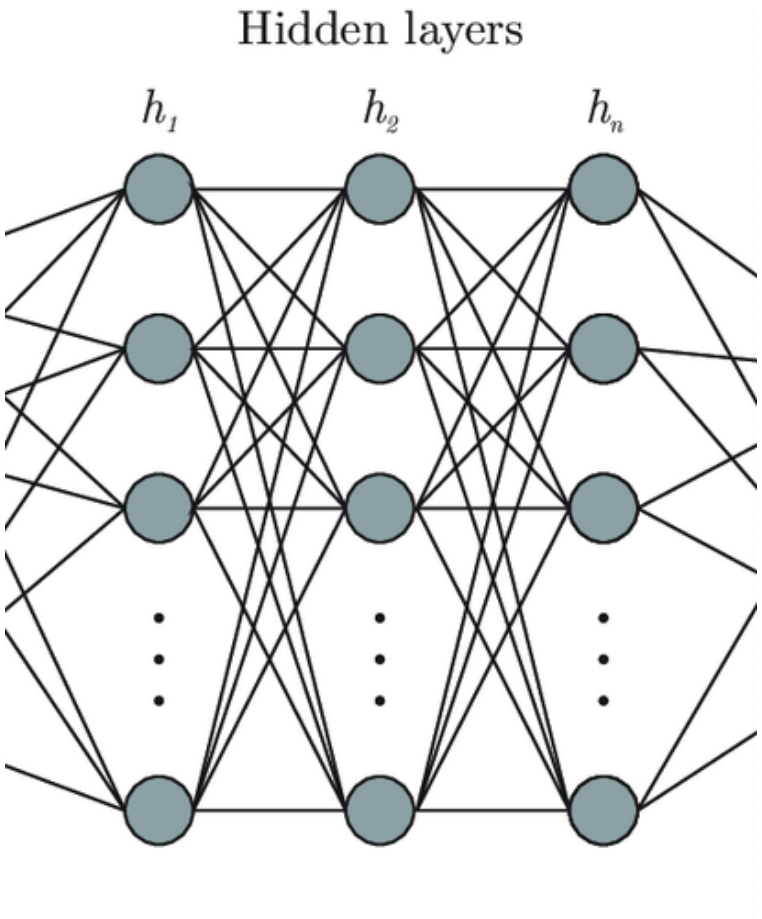

Input layer
Hidden layers
Output layer
i
h_1
h_2
h_n
o
Input 1
Input 2
Input n
Output 1
Output n

Input Layer

- Layer pertama dalam jaringan saraf yang menerima input dari luar dan memberikan output ke lapisan berikutnya dalam jaringan.
- Lapisan input terdiri dari beberapa neuron, tergantung pada jumlah atribut atau fitur dalam input.
- Setiap neuron pada input layer mewakili satu atribut atau fitur dari input dan menerima input numerik yang sesuai dengan nilai atribut.



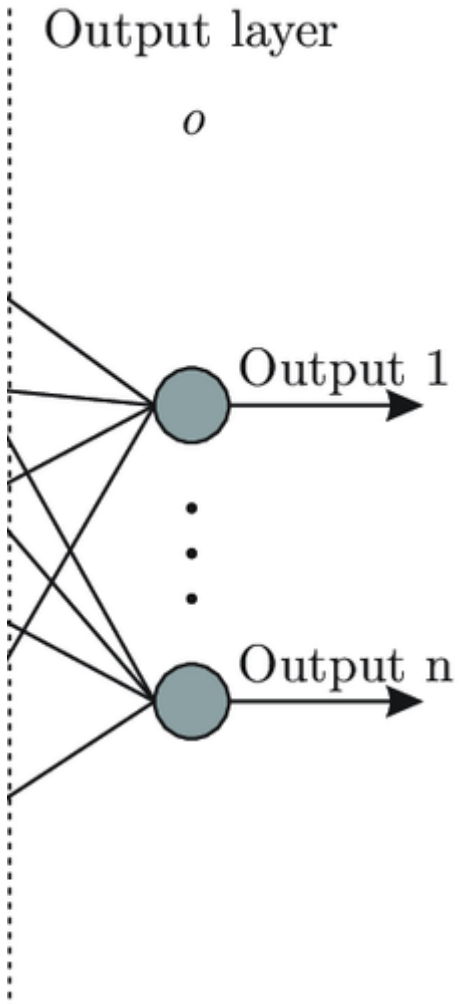
Hidden Layer



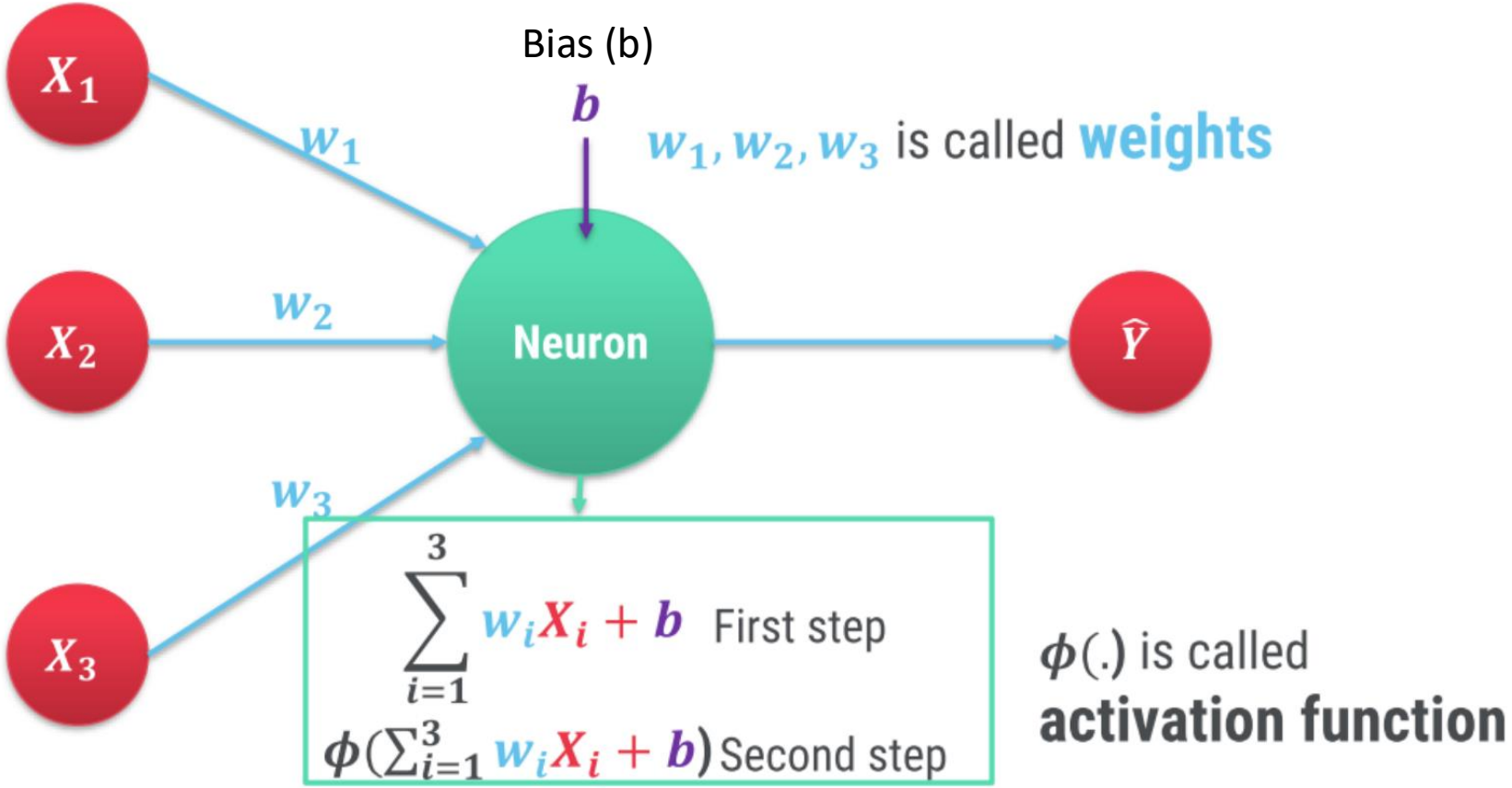
- Layer di antara input layer dan output layer yang terdiri dari beberapa neuron yang saling terhubung dan masing-masing neuron melakukan pemrosesan terhadap input yang diterima dari lapisan sebelumnya dan memberikan output ke lapisan berikutnya.
- Fungsi utama dari hidden layer adalah untuk mengekstraksi fitur atau pola dari data input yang kompleks dan membuat jaringan saraf lebih fleksibel dalam memproses data.
- Pemilihan jumlah neuron pada setiap hidden layer dan jumlah hidden layer pada jaringan saraf adalah tugas yang penting dalam membangun model jaringan saraf yang efektif.
- Terlalu sedikit neuron atau hidden layer dapat menghasilkan model yang kurang akurat, sedangkan terlalu banyak dapat menyebabkan overfitting.

Output Layer

- Layer terakhir dalam jaringan saraf yang menghasilkan output.
- Bentuk dan fungsi dari output layer tergantung pada jenis masalah yang ingin diselesaikan.
- Pada masalah klasifikasi, output layer biasanya terdiri dari beberapa neuron, di mana setiap neuron mewakili kelas yang mungkin dari input.
- Pada masalah regresi, output layer biasanya terdiri dari satu neuron, yang menghasilkan nilai numerik yang merepresentasikan prediksi dari model terhadap input yang diberikan.
- Nilai output dari setiap neuron pada output layer dihitung dengan menggunakan fungsi aktivasi yang sesuai dengan jenis masalah yang ingin diselesaikan.



Proses Training pada ANN



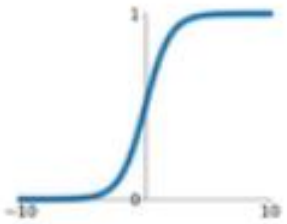
Activation Function

- Fungsi matematika yang diterapkan pada setiap neuron pada ANN untuk menghasilkan output dari nilai input yang diberikan.
- Fungsi ini memungkinkan jaringan saraf untuk mempelajari dan mengekspresikan relasi antara input dan output pada data.
- Terdapat berbagai jenis fungsi aktivasi yang umum digunakan pada jaringan saraf buatan, antara lain:
  - Fungsi Sigmoid
  - ReLU (Rectified Linear Unit)
  - Tanh (Tangent Hyperbolic)
  - Maxout
  - dll

Some Activation Functions

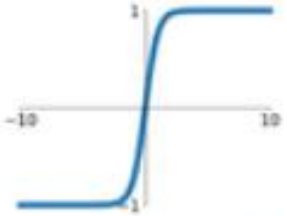
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



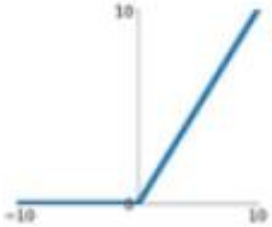
tanh

$\tanh(x)$



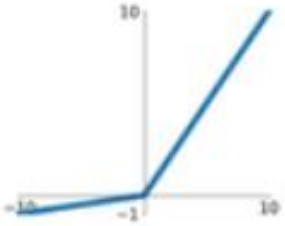
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$

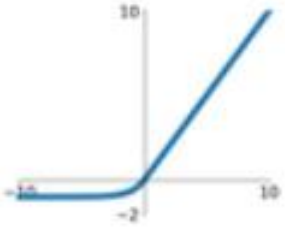


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Proses Training pada ANN (Fitting)

- Proses training (pemodelan) pada neural network dilakukan untuk mendapatkan bobot (w_i) terbaik.
- Bobot (w_i) terbaik didapatkan dengan meminimumkan suatu loss function dengan algoritma optimisasi tertentu.
- Banyaknya iterasi (ulangan) yang dilakukan untuk meng-update bobot menggunakan seluruh training data disebut epoch.
- Proses update bobot dilakukan dengan algoritma backpropagation.
- Kecepatan konvergensi proses train bisa diatur dengan learning rate yang nilainya antara 0 dan 1.
- Semakin kecil learning rate, maka epoch yang dibutuhkan akan semakin banyak.

Step1: Inisiasi Bobot (weight)

- Bobot numerik yang mewakili kekuatan koneksi antara dua neuron atau antara neuron dan input.
- Weight merupakan parameter penting pada ANN yang menentukan seberapa besar pengaruh setiap input pada output yang dihasilkan oleh neuron.
- Setiap input pada sebuah neuron dikalikan dengan bobot tertentu sebelum dijumlahkan dan diproses melalui fungsi aktivasi.

Peristilahan: Bias

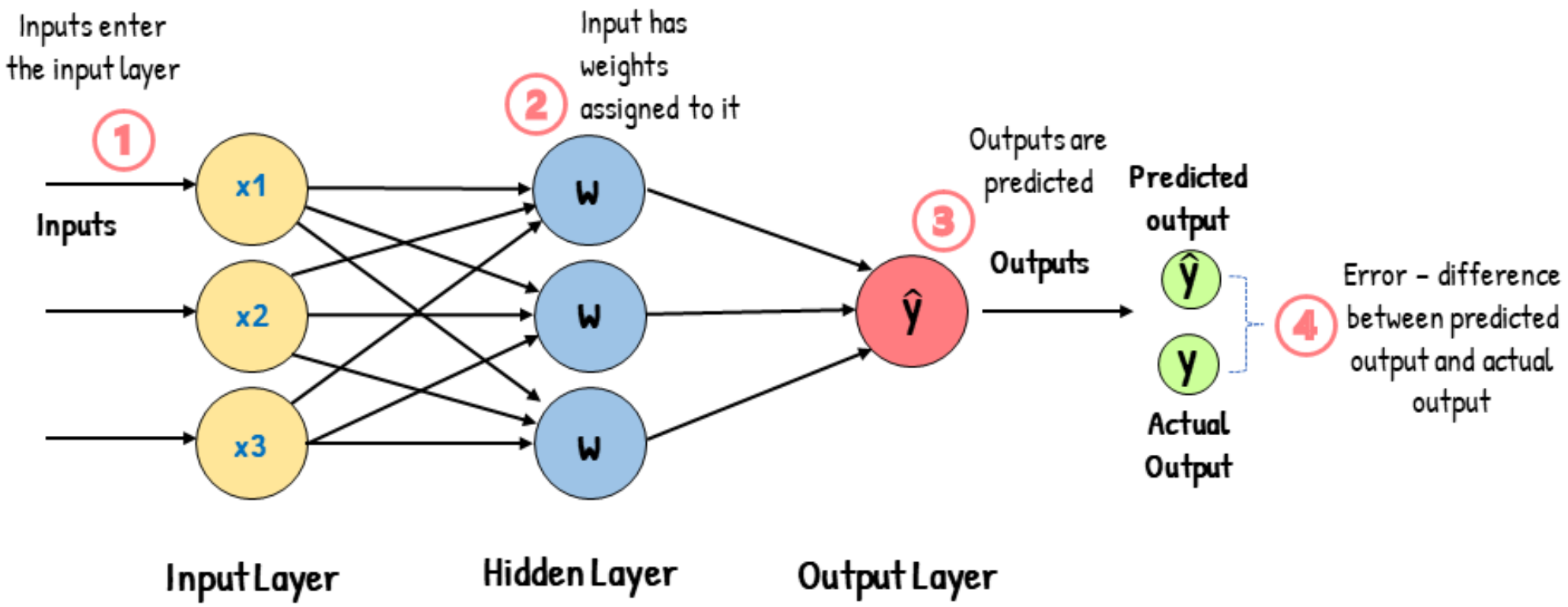
- Parameter tambahan pada setiap neuron dalam ANN yang berfungsi untuk menambahkan nilai tetap pada setiap input sebelum dihitung melalui fungsi aktivasi pada neuron tersebut.
- Bias pada setiap neuron memungkinkan jaringan saraf untuk dapat memodelkan relasi antara input dan output dengan lebih fleksibel, karena tanpa adanya bias, jaringan saraf hanya akan dapat menghasilkan garis lurus atau linear pada plot input-output.
- Setiap neuron pada jaringan saraf buatan memiliki satu parameter bias.

Step 2: Feedforward

Feedforward adalah proses propagasi maju di mana input melewati lapisan-lapisan jaringan untuk menghasilkan output.

1. Input diberikan ke setiap neuron di **input layer**.
2. Setiap neuron menghitung kombinasi linier dari input dengan bobot dan bias.
3. **Fungsi aktivasi** diterapkan untuk menghasilkan nilai keluaran yang akan diteruskan ke lapisan berikutnya.
4. Proses ini berlanjut hingga mencapai **output layer**, yang memberikan hasil akhir prediksi.

Feed-Forward Neural Network

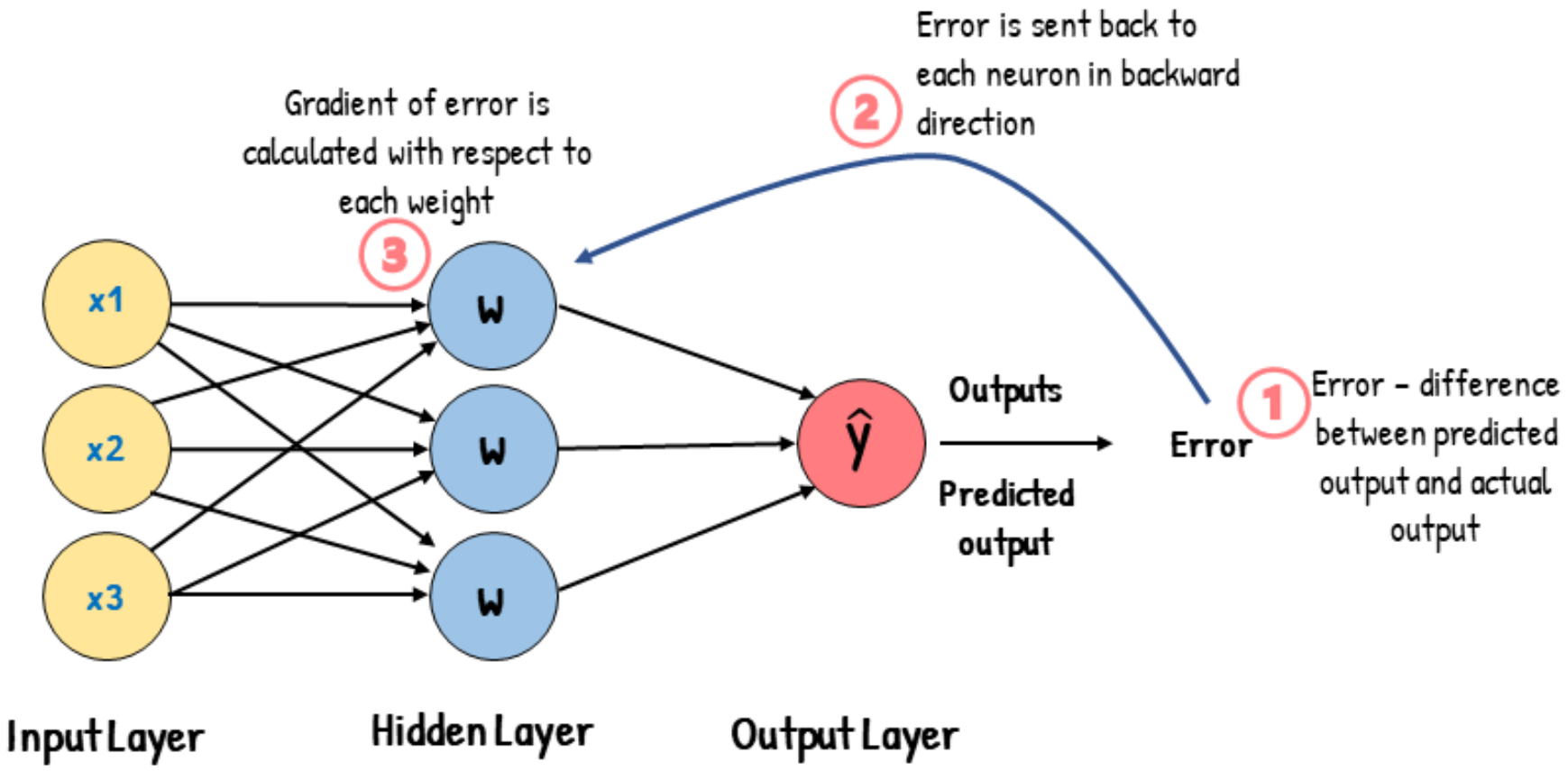


Step 3: Backpropagation

Backpropagation adalah algoritma optimasi yang digunakan untuk menyesuaikan bobot dalam jaringan saraf agar dapat meminimalkan error.

1. **Perhitungan error**: Selisih antara output yang dihasilkan dan target yang diharapkan dihitung menggunakan fungsi loss seperti Mean Squared Error (MSE) atau Cross-Entropy.
2. **Backward pass**: Error disebarkan kembali ke belakang untuk menghitung gradien dari setiap bobot.
3. **Update bobot**: Bobot diperbarui menggunakan metode optimasi seperti **Gradient Descent**: dengan **η** adalah learning rate dan **E** adalah error total.
4. Proses ini diulang hingga error minimum tercapai.

Backpropagation



Pehitungan Error dalam Backpropagation

- Untuk Regresi

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- Untuk Klasifikasi

$$L = -\sum y \log(\hat{y})$$

Perhitungan Gradient Descent

- Menghitung Gradien Terhadap Bobot:

$$\frac{\partial L}{\partial W} = -2(y - \hat{y})X$$

- Update Bobot dengan Gradient Descent

$$W = W - \eta \frac{\partial L}{\partial W}$$

$$b = b - \eta \frac{\partial L}{\partial W}$$

Step 4: Ulangi

- Ulangi hingga tidak ada perubahan nilai error, nilai error kurang dari atau sama dengan suatu threshold (biasanya mendekati 0), atau sudah mengulangi proses latihan sebanyak T kali (threshold).

Contoh Kasus: Prediksi Kelulusan Siswa

Terdapat 10 siswa yang memiliki nilai UTS dan UAS sbb:

Mahasiswa	UTS	UAS	Lulus
A	80	90	1
B	70	85	1
C	60	75	0
D	90	95	1
E	50	65	0
F	85	80	1
G	45	55	0
H	77	85	1
I	65	70	0
J	95	98	1

Step 1: Inisisasi Bobot

- Inisasi Bobot Awal
- Misal w1 = 0.3, w2 = 0.5
- Bias (b) = -10

Step 2: Feedforward

- Hitung hasil transformasi linier untuk mahasiswa

$$Z = (w_1 * x_1) + (w_2 * x_2) + b$$

- Contoh: Mahasiswa A

$$Z_A = (0.3 * 80) + (0.5 * 90) - 10 = 24$$

- Sebagai contoh, gunakan fungsi aplikasi sigmoid:

$$\phi = \sigma(Z_A) = \frac{1}{1 + e^{-Z}} = \frac{1}{1 + e^{-24}} \approx 1$$

Step 3: Backpropagation

- Hitung Loss dengan MSE:

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- Misalkan hasil prediksi untuk 10 mahasiswa adalah [1, 0.98, 0.6, 1, 0.35, 1, 0.20, 0.9, 0.5, 0.1]
  maka:

$$L = \frac{1}{10}[(1-1)^2 + (1-0.98)^2 + \cdots + (1-0.1)^2] = 0.051$$

Step 3: Backpropagation

- Menghitung Gradien Terhadap Bobot. Contoh untuk mahasiswa C:
- UTS 60, UAS 75
- $y = 0;\ \hat{y} = 0.6$

$$\frac{\partial L}{\partial W_1} = -2(0 - 0.6)\ 60 = 72; \frac{\partial L}{\partial W_2} = -2(0 - 0.6)\ 75 = 90$$

- Update Bobot dengan Gradient Descent (misal $\eta = 0.1$ sbg learning rate)

$$W_1{}' = 0.3 - (0.1 \times 72) = -6.9; W_2{}' = 0.5 - (0.1 \times 90) = -8.5$$

- Selanjutnya bobot tsb diperbaharui utk iterasi berikutnya
- Proses ini diulang hingga loss mencapai nilai minimum atau batas iterasi tercapai.

Contoh di R: Kasus Klasifikasi Dataset Iris (1)

```
# Memuat pustaka yang diperlukan
library(nnet)

# Memuat dataset iris
data(iris)

# Mengonversi label target menjadi numerik
iris$Species <- as.numeric(iris$Species) - 1

# Membagi data menjadi training dan testing
train_idx <- sample(1:nrow(iris), 0.8 * nrow(iris)) # Memilih 80% data
sebagai training
train_data <- iris[train_idx, ]  # Data training
test_data <- iris[-train_idx, ]  # Data testing
```

Contoh di R: Kasus Klasifikasi Dataset Iris (2)

```
# Membuat model Neural Network
nn_model <- nnet(Species ~ ., data=train_data, size=5, decay=0.01, maxit=200)
# size=5: Jumlah neuron dalam hidden layer
# decay=0.01: Regularisasi untuk menghindari overfitting
# maxit=200: Jumlah iterasi maksimum

# Melakukan prediksi pada data testing
predictions <- predict(nn_model, test_data[, -5], type="class")

# Menampilkan hasil prediksi dalam bentuk tabel perbandingan dengan label asli
table(predictions, test_data$Species)
```

Contoh di R: Kasus Regresi Dataset Boston Housing (1)

```
# Memuat pustaka yang diperlukan
library(MASS)       # Memuat dataset Boston
library(neuralnet)  # Pustaka untuk Neural Network

# Memuat dataset Boston Housing
data(Boston)

# Melakukan normalisasi dataset agar model dapat bekerja lebih optimal
boston_data <- as.data.frame(scale(Boston))

# Membagi data menjadi training dan testing
train_idx <- sample(1:nrow(boston_data), 0.8 * nrow(boston_data)) # 80% untuk training
train_data <- boston_data[train_idx, ]  # Data training
test_data <- boston_data[-train_idx, ]  # Data testing
```

Contoh di R: Kasus Regresi Dataset Boston Housing (2)

```
# Membuat model Neural Network
nn_model <- neuralnet(medv ~ ., data=train_data, hidden=c(5,3), linear.output=TRUE)
# hidden=c(5,3): Dua hidden layer, dengan masing-masing 5 dan 3 neuron
# linear.output=TRUE: Menggunakan output linear untuk regresi

# Melakukan prediksi pada data testing
predictions <- compute(nn_model, test_data[, -14])$net.result

# Menampilkan hasil prediksi
head(predictions)
```

Terima Kasih